

30 Bài Tập Array List & Linked List

PHẦN A — ARRAY LIST (DANH SÁCH MẢNG ĐỘNG) · 15 bài

Bài 1. Cài đặt Array List cơ bản

Cài đặt danh sách mảng động với `add`, `get`, `set`, `size`. Truy cập phần tử theo chỉ số trong **$O(1)$** .

Ví dụ: `add 1,2,3 ; get(1) → 2`

Bài 2. Thêm / xóa ở cuối

Cài đặt `append` (thêm cuối) và `popBack` (xóa cuối). Cả hai chạy **$O(1)$** (chưa tính mở rộng).

Ví dụ: `[1,2,3] popBack → [1,2]`, trả về 3

Bài 3. Chèn / xóa ở vị trí bất kỳ

Chèn hoặc xóa phần tử tại chỉ số `i` bằng cách **dịch chuyển** các phần tử phía sau. Phân tích chi phí $O(n)$.

Ví dụ: `[1,2,4]` chèn 3 tại `idx 2` → `[1,2,3,4]`

Bài 4. Tìm kiếm tuyến tính

Trả về chỉ số đầu tiên của một giá trị (`indexOf`), hoặc -1 nếu không có.

Ví dụ: `[5,3,7]`, tìm 7 → 2

Bài 5. Duyệt và in phần tử

Duyệt toàn bộ danh sách, in các phần tử; đếm số phần tử thỏa một điều kiện.

Ví dụ: `[1,2,3,4]` → đếm số chẵn = 2

Bài 6. Tự động mở rộng dung lượng

Khi mảng đầy, cấp phát mảng mới **gấp đôi** dung lượng và sao chép phần tử sang. Cài đặt cơ chế `resizing` này.

Ví dụ: đầy ở `cap 4` → cấp phát `cap 8`

Bài 7. Phân tích amortized của append

Chứng minh thao tác `append` có chi phí **trung bình (amortized) $O(1)$** dù thỉnh thoảng phải sao chép $O(n)$ khi mở rộng.

Ví dụ: `n` lần `append` → tổng $O(n)$ thao tác

Bài 8. Xóa các phần tử thỏa điều kiện

Xóa **tại chỗ** mọi phần tử thỏa một điều kiện (`removeIf`) và giữ nguyên thứ tự các phần tử còn lại, trong **$O(n)$** (kỹ thuật hai con trỏ đọc/ghi).

Ví dụ: `[1,2,3,4]` xóa số chẵn → `[1,3]`

Bài 9. Đảo ngược tại chỗ

Đảo ngược thứ tự các phần tử **tại chỗ** dùng hai con trỏ hai đầu, $O(n)$ thời gian, $O(1)$ bộ nhớ.

Ví dụ: `[1,2,3,4]` → `[4,3,2,1]`

Bài 10. Trộn hai danh sách đã sắp xếp

Trộn hai array list đã sắp xếp thành một danh sách sắp xếp, dùng hai con trỏ.

Ví dụ: $[1, 3, 5] + [2, 4] \rightarrow [1, 2, 3, 4, 5]$

Bài 11. Xoay mảng k vị trí

Xoay danh sách sang phải k vị trí **tại chỗ**. Gợi ý: kỹ thuật đảo ba lần (reverse).

Ví dụ: $[1, 2, 3, 4, 5], k=2 \rightarrow [4, 5, 1, 2, 3]$

Bài 12. Loại bỏ trùng lặp giữ thứ tự

Loại bỏ các phần tử lặp, **giữ lần xuất hiện đầu tiên** theo đúng thứ tự. So sánh cách $O(n^2)$ và cách $O(n)$ dùng tập băm.

Ví dụ: $[3, 1, 3, 2, 1] \rightarrow [3, 1, 2]$

Bài 13. Trộn các khoảng (Merge Intervals)

Cho danh sách các khoảng $[start, end]$, **gộp các khoảng giao nhau**. Gợi ý: sắp xếp theo start rồi quét một lượt.

Ví dụ: $[[1, 3], [2, 6], [8, 10]] \rightarrow [[1, 6], [8, 10]]$

Bài 14. Mảng động 2 chiều

Cài đặt một danh sách 2 chiều (ma trận động) cho phép thêm hàng/cột linh hoạt và truy cập phần tử (i, j) trong $O(1)$.

Ví dụ: thêm hàng, `set(i, j, val)`, `get(i, j)`

Bài 15. Iterator và fail-fast

Cài đặt một **iterator** cho array list. Mở rộng để phát hiện **sửa đổi cấu trúc trong khi đang duyệt** (fail-fast) bằng biến đếm modCount.

Ví dụ: sửa list khi đang lặp $\rightarrow$ báo lỗi

PHẦN B — LINKED LIST (DANH SÁCH LIÊN KẾT) · 15 bài

Bài 1. Cài đặt danh sách liên kết đơn

Cài đặt singly linked list với thao tác thêm đầu (`pushFront`), thêm cuối (`pushBack`) và in toàn bộ.

Ví dụ: `pushFront 2, pushBack 5` $\rightarrow 2 \rightarrow 5 \rightarrow \text{null}$

Bài 2. Tính độ dài / duyệt

Duyệt danh sách và đếm số nút; in giá trị các nút theo thứ tự.

Ví dụ: `1` $\rightarrow$ `2` $\rightarrow$ `3` $\rightarrow$ `null` $\rightarrow$ độ dài 3

Bài 3. Tìm kiếm một giá trị

Tìm xem một giá trị có trong danh sách không; trả về vị trí (chỉ số nút) nếu có.

Ví dụ: `1` $\rightarrow$ `2` $\rightarrow$ `3`, tìm 2 $\rightarrow$ vị trí 1

Bài 4. Chèn sau một nút cho trước

Chèn một nút mới **ngay sau** một nút đã cho (hoặc sau vị trí thứ k).

Ví dụ: `1` $\rightarrow$ `3` $\rightarrow$ `null`, chèn 2 sau nút 1 $\rightarrow 1 \rightarrow 2 \rightarrow 3$

Bài 5. Xóa nút theo giá trị

Xóa nút đầu tiên có giá trị bằng x (xử lý cả trường hợp xóa nút đầu).

Ví dụ: $1 \rightarrow 2 \rightarrow 3 \rightarrow 2$, xóa 2 $\rightarrow 1 \rightarrow 3 \rightarrow 2$

Bài 6. Đảo ngược danh sách liên kết

Đảo ngược một danh sách liên kết đơn bằng cả hai cách: **lặp** (ba con trỏ) và **đệ quy**.

Ví dụ: $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{null} \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{null}$

Bài 7. Tìm nút giữa

Tìm nút giữa của danh sách dùng kỹ thuật **con trỏ nhanh/chậm** (slow/fast) trong một lượt.

Ví dụ: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow$ nút giữa = 3

Bài 8. Phát hiện chu trình

Kiểm tra danh sách có **chu trình** (vòng lặp) hay không bằng thuật toán Floyd (rùa và thỏ).

Ví dụ: có nút cuối trỏ về giữa $\rightarrow$ có chu trình

Bài 9. Trộn hai danh sách đã sắp xếp

Trộn hai danh sách liên kết đã sắp xếp thành một danh sách sắp xếp (nối lại các nút, không tạo nút mới).

Ví dụ: $1 \rightarrow 3 \rightarrow 5$ và $2 \rightarrow 4 \rightarrow$ $\rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Bài 10. Xóa nút thứ k từ cuối

Xóa nút thứ k tính từ cuối danh sách trong **một lượt duyệt** (hai con trỏ cách nhau k bước).

Ví dụ: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$, $k=2 \rightarrow$ xóa 4

Bài 11. Danh sách liên kết đôi

Cài đặt doubly linked list cho phép thêm/xóa ở cả hai đầu và duyệt hai chiều (mỗi nút có con trỏ prev và next).

Ví dụ: thêm đầu/cuối, duyệt xuôi và ngược

Bài 12. Tìm điểm bắt đầu chu trình

Khi danh sách có chu trình, tìm **nút bắt đầu** của chu trình (giai đoạn 2 của thuật toán Floyd).

Ví dụ: trả về nút nơi vòng lặp khép lại

Bài 13. Sắp xếp danh sách liên kết

Sắp xếp một danh sách liên kết trong **$O(n \log n)$** bằng merge sort (tách đôi bằng slow/fast, trộn đệ quy).

Ví dụ: $3 \rightarrow 1 \rightarrow 2 \rightarrow \text{null} \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{null}$

Bài 14. Cộng hai số biểu diễn bằng linked list

Hai số nguyên được lưu ngược trong hai danh sách (mỗi nút một chữ số). Tính tổng và trả về danh sách kết quả, xử lý nhớ (carry).

Ví dụ: $(2 \rightarrow 4 \rightarrow 3) + (5 \rightarrow 6 \rightarrow 4) \rightarrow 7 \rightarrow 0 \rightarrow 8$ ($342+465=807$)

Bài 15. LRU Cache

Thiết kế **LRU Cache** dung lượng cố định với get và put đều **$O(1)$** , dùng **doubly linked list + bảng băm**.

Ví dụ: cache cap=2, truy cập đầy phần tử mới nhất lên đầu